Atina JDE Edwards Microservice - Installation Guide

Table of Contents

Introduction

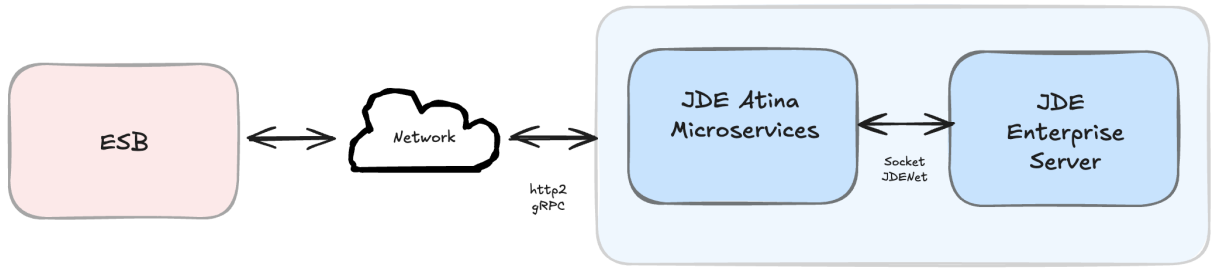
What Are Atina JDE Microservices?

Atina JDE Microservices are Docker-based components designed to interact directly with Oracle's *JD Edwards EnterpriseOne™*. They serve as a modern intermediary that improves upon legacy integration methods.

Why Use Them?

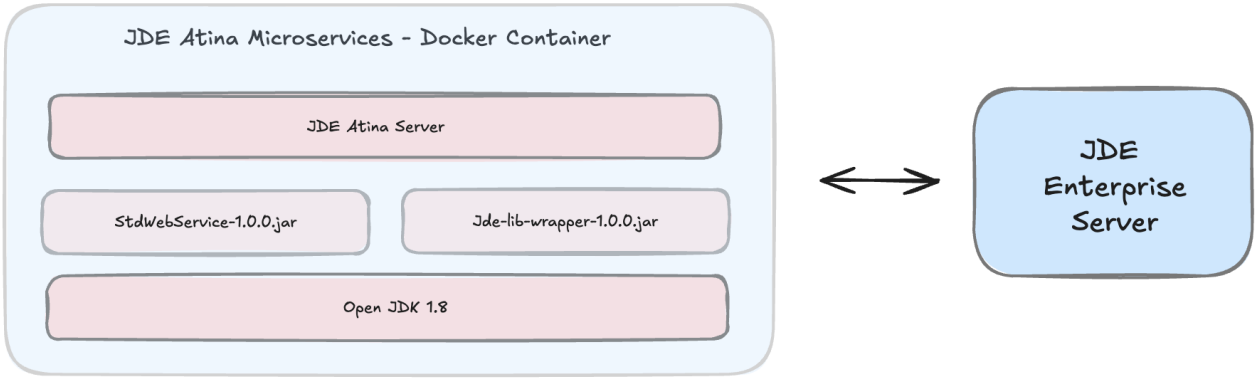
By running inside the client's network, these microservices overcome key issues commonly found in WAN or cloud-based integrations—particularly those stemming from the use of socket-based communication in the original JDE libraries. This deployment approach improves performance, reduces latency, and avoids DNS resolution problems.

Furthermore, the microservices expose a gRPC-based API (using HTTP/2), eliminating the dependency on Java 8-only Oracle libraries. This ensures compatibility with newer Java versions (such as Java 17), aligning with *MuleSoft™*'s roadmap of deprecating older Java support.



How is implemented?

The microservice is packaged as a Docker container and runs using Java 8. It communicates directly with the JD Edwards Enterprise Server through Oracle's interoperability layer and exposes its functionality via a gRPC-based API using the HTTP/2 protocol. This design ensures modern performance while maintaining backward compatibility with JDE tooling.

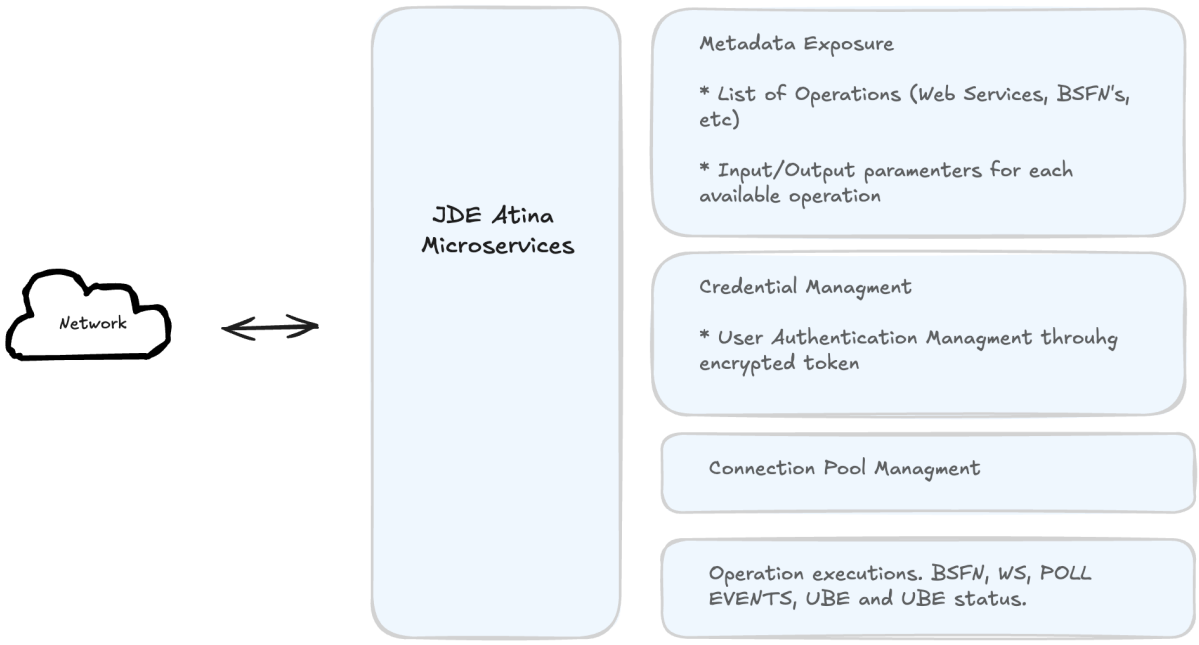


Key Functionalities

The *Atina JDE Microservices* component specifically provides access to several *JD Edwards EnterpriseOne™* operations, managing credentials, sessions, connection pooling, and metadata introspection.

Functionally, the *Atina JDE Microservices* enable the connector for several *JD Edwards EnterpriseOne™* key operations:

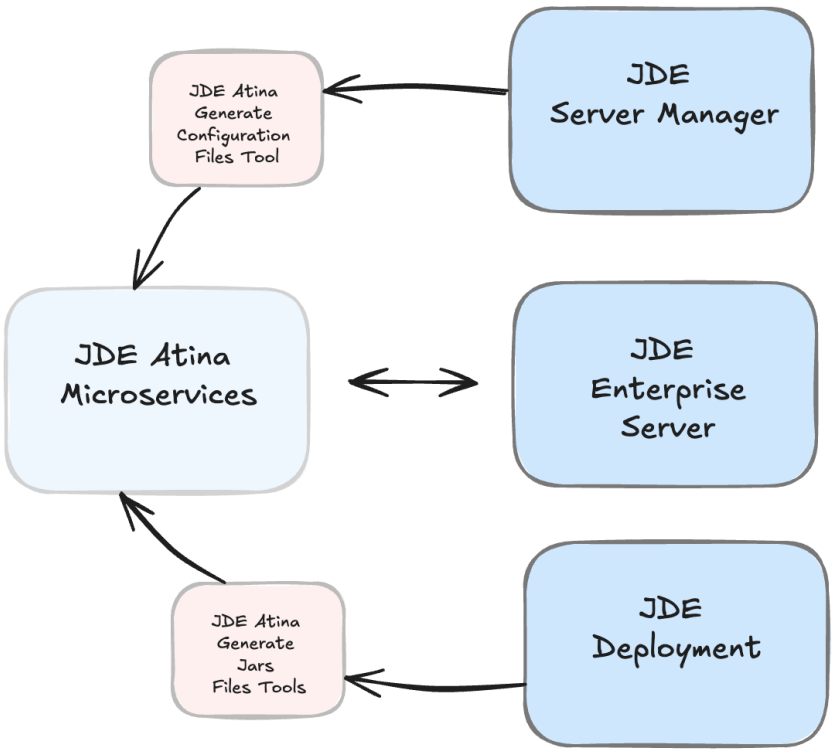
- Authentication methods, including Log in *JD Edwards EnterpriseOne™* with JDE User Credentials or Tokens.
- Invoke *JD Edwards EnterpriseOne™* web services.
- Submitting UBE by name and version processes, including data selection to filter records, override Processing Options and passing Report Interconnect (RI) parameter. The user can override the Batch Queue too.
- Check a UBE call's status asynchronously.
- Invoking Business Functions (BSFNs) on the JDE server with parameters and Transaction Processing handling.
- Polling for Master Business Function (MBF) events
- Metadata APIs to retrieve lists of available web services, input/output parameters, and payload templates.
- Monitoring APIs to view open connections and retrieve logs by transaction ID
- Token APIs for creating and parsing tokens.



Tooling Provided

The Atina JDE Microservices suite also includes additional tools to simplify configuration and deployment, such as:

- JD Atina Generate Configuration Files Tool: Helps generate base .ini files (like jdbj.ini, jdeinterop.ini, jdelog.properties) by extracting information from JDE Enterprise Server Manager via its REST API.
- JD Atina Generate Jars Files Tool: Generates necessary JAR files, including jde-lib-wrapped-1.0.0.jar and StdWebService-1.0.0.jar



Overall, the *Atina JDE Microservices* provide a robust and modernized solution for integrating *JD Edwards EnterpriseOne™* with other systems, addressing key technical challenges and aligning with contemporary integration platform requirements

Prerequisites

This guide assumes that you have a working knowledge of the following topics:

- Oracle's *JD Edwards EnterpriseOne™* interoperability.
- Oracle's *JD Edwards EnterpriseOne™* Network Connectivity.
- Dockers basis.
- Maven basis.

Software Requirements

To install and run the Atina JDE Microservices and accompanying tools, you will need the following software:

- **Docker** — Required to deploy the microservices, which are containerized.
- **Git** — Used to download configuration and tooling repositories.
- **Java Development Kit (JDK) 8** — Required if you intend to run the configuration or JAR generation tools as standalone Java applications.
- **Maven 3.8.1 or higher** — Required for managing dependencies and building artifacts.

Oracle's *JD Edwards EnterpriseOne*™ Requirements

The following resources and configurations must be available in your JD Edwards environment:

- **Access to local JD Edwards Java libraries** — These libraries must be extracted from your JD Edwards Deployment Server.
- **INI configuration review (jde.ini)** — The microservice builds upon Oracle's interoperability architecture, which usually works without modification. However, you must verify the `XMLLookupInfo` section in `jde.ini`, particularly the `XMLRequestType7=ube` entry, as this setting is often required for UBE submissions. The following XML kernel sections are usually configured correctly by default:
  - XML Dispatch Kernel
  - XML CallObject Kernel
  - XML Transaction Kernel
  - XML List Kernel
- **Firewall considerations (JDENET Ports)** — Enable `PredefinedJDENETPorts` in `jde.ini` **only** if there is a firewall between the JD Edwards Enterprise Server and your integration layer. If the microservices are deployed within the same network (as recommended), this setting is not necessary.
- **Oracle *JD Edwards EnterpriseOne* Server Manager credentials**: Admin user, password, and the Server Manager URL (e.g. `http://server:8999/manage/`).
- **Java libraries on Deployment Server folders**. Make sure you include the following directories from your JDE Deployment Server in your project's resources:
  - `//Deployment Server/E920/MISC`
  - `//Deployment Server/E920/system/Classes`
  - `//Deployment Server/E920/system/JAS/webclient.ear/webclient.war/WEB-INF/lib`
  - `//Deployment Server/E920/DV920/java/sbfjars`

NOTE	This configuration must be completed or validated by your CNC administrator. Refer to the *JD Edwards EnterpriseOne™ Tools Interoperability Guide* for further details.

Compatibility

Oracle's *JD Edwards EnterpriseOne*™ Tools Release 8.98 Update 4 and onwards. For Web Services we recomend Tool Release 9.2.x and onwards due limitations for previous versions.

Preliminary Setup

To ensure the microservice operates correctly, you must complete the following preparatory steps:

1. **Configure required INI files** Set up the standard *JD Edwards EnterpriseOne*™ interoperability

INI files (jdeinterop.ini, jdbj.ini, jdelog.properties, tnsnames.ora).

2. **Generate libraries** Package the required JARs from your Deployment Server into two reusable bundles using the provided tooling.
3. **Verify JDE Server Configuration** Review and (if necessary) adjust the `XMLLookupInfo` section in your jde.ini to enable UBE submission. Other kernel settings (XML Dispatch, CallObject, Transaction, List) are typically correct out of the box.
4. **Install the Atina JDE Microservice** Install and configure the microservice container in your environment using the Docker setup instructions.

Configure required INI files

JDE Atina Microservice needs the following ini files according to your environment:

- jdbj.ini
- jdeinterop.ini
- jdelog.properties
- tnsnames.ora" (for Oracle RDBMS based installations only)

You can configure it manually following JDE manuals, or you can use **JDE Atina Generate Configuration Files Tool** to automatically extracts and stages the required INI files.

JDE Atina Generate Configuration Files Tool help to you to generate a base ini files. It takes all information from JDE Enterprise Server Manager using REST API for Server Manager. This tool will require to select the HTML Client instance according to your environment.

We have created a special section where we explain how to use this tool.

NOTE	At the end of this process, We recommend review jdeinterop.ini and jdbj.ini files before deploy it on JD Atina Web Service Microservice using this guide: Understanding jdeinterop.ini File.

JDELOG.PROPERTIES (optional)

NOTE : See *JD Edwards EnterpriseOne*™ documentation for usage

```
[E1LOG]
FILE=/tmp/jdelog/jderoot.log
LEVEL=SEVERE
FORMAT=APPS
MAXFILESIZE=10MB
MAXBACKUPINDEX=20
COMPONENT=ALL
APPEND=TRUE

#Logging runtime and JAS above APP level will be helpful for application developers.
#Application developers should use this log as a substitute to analyze the flow of
events
```

```
#in the webclient.
[JASLOG]
FILE=/tmp/jdelog/jas.log
LEVEL=APP
FORMAT=APPS
MAXFILESIZE=10MB
MAXBACKUPINDEX=20
COMPONENT=RUNTIME|JAS|JDBJ
APPEND=TRUE

#Logging runtime and JAS at DEBUG level will be helpful for tools developers.
#Tool developers should use this log ato debug tool level issues
[JASDEBUG]
FILE=/tmp/jdelog/jasdebug.log
LEVEL=DEBUG
FORMAT=TOOLS_THREAD
MAXFILESIZE=10MB
MAXBACKUPINDEX=20
COMPONENT=ALL
APPEND=TRUE
```

Generate libraries

JDE Atina Microservice makes use of the libraries of the local JDE Tools release. In order to simplify dependency management for the microservice application, we need to package JDE libraries together into a lib bundle using the provided **JDE Atina Generate Jars Files Tool**.

This tool will generate two jars files need it by JD Web Service Microservice.

Because it is a complex process, we have created an application that makes it easy to create.

Verify JDE Enterprise Server configuration

JD Edwards EnterpriseOne™ Server configuration requirements

To ensure correct operation of all of the JDE Connector features the Enterprise Server requires the following jde.ini file settings:

Please refer to **JD Edwards EnterpriseOne Tools Interoperability Guide** to check updates, and also provides the different .dll extensions for other platforms.

NOTE	The below .dll files, all relate to the *Microsoft Windows* platform.
IMPORTANT	This configuration must be done by CNC administrator. Refer to **JD Edwards EnterpriseOne Tools Interoperability Guide**

1. Ensure that sufficient processes are available for the **XML List Kernel**

```
[JDENET_KERNEL_DEF16]

krnlName=XML List Kernel
dispatchDLLName=xmllist.dll
dispatchDLLFunction=_XMLListDispatch@28
maxNumberOfProcesses=3
numberOfAutoStartProcesses=1
```

2. Ensure that sufficient processes are available for the **XML Dispatch Kernel**

```
[JDENET_KERNEL_DEF22]
dispatchDLLName=xmldispatch.dll
dispatchDLLFunction=_XMLDispatch@28
maxNumberOfProcesses=1
numberOfAutoStartProcesses=1
```

3. Ensure that sufficient processes are available for the **XML Service Kernel**

```
[JDENET_KERNEL_DEF24]
krnlName=XML Service KERNEL
dispatchDLLName=xmlservice.dll
dispatchDLLFunction=_XMLServiceDispatch@28
maxNumberOfProcesses=1
numberOfAutoStartProcesses=0
```

4. Ensure that the **LREngine** has a suitable output storage location and sufficient disk allocation

```
[LREngine]
System=C:\JDEdwardsPPack\E920\output
Repository_Size=20
Disk_Monitor=YES
```

5. Ensure that the XML Kernels are correctly defined

```
[XMLLookupInfo]
XMLRequestType1=list
XMLKernelMessageRange1=5257
XMLKernelHostName1=local
XMLKernelPort1=0

XMLRequestType2=callmethod
XMLKernelMessageRange2=920
XMLKernelHostName2=local
XMLKernelPort2=0
```

```
XMLRequestType3=trans
XMLKernelMessageRange3=5001
XMLKernelHostName3=local
XMLKernelPort3=0

XMLRequestType4=JDEMSGWFINTEROP
XMLKernelMessageRange4=4003
XMLKernelHostName4=local
XMLKernelPort4=0
XMLKernelReply4=0

XMLRequestType5=xapicallmethod
XMLKernelMessageRange5=14251
XMLKernelHostName5=local
XMLKernelPort5=0
XMLKernelReply5=0

XMLRequestType6=realTimeEvent
XMLKernelMessageRange6=14251
XMLKernelHostName6=local
XMLKernelPort6=0
XMLKernelReply6=0

XMLRequestType7=ube
XMLKernelHostName7=local
XMLKernelMessageRange7=380
XMLKernelPort7=0
XMLKernelReply7=1
```

Enterprise Server connection considerations

- Enable Predefined JDENET Ports in JDE.INI

When there is a firewall between the Mulesoft ESB and the Enterprise Server, set the PredfinedJDENETPorts setting to 1 in the JDE.INI file of the Enterprise Server. This setting enables JDENET network process to use a predefined range of TCP/IP ports. This port range starts at the port number that is specified by serviceNameListen and ends at the port that is calculated by the equation serviceNameListen = maxNetProcesses - 1. You must open these ports in a firewall setup to successfully connect the Mulesoft ESB to the Enterprise Server.

Please refer to **JD Edwards EnterpriseOne Tools Security Administration Guide** to check updates.

JDE Atina Generate Configuration Files Tool

This tool simplifies the generation of JD Edwards interoperability configuration files required by the microservice. It connects to the JD Edwards Server Manager via REST API and automatically extracts settings for a selected HTML instance.

You can use it in two ways:

- As a Java CLI application (requires JDK 8)
- As a Docker container

The following files will be generated:

- `jdbj.ini`
- `jdeinterop.ini`
- `jdelog.properties`
- `settings.xml`

If your environment uses Oracle RDBMS, you must also add the `tnsnames.ora` file manually.

NOTE	After generation, review the contents of `jdbj.ini` and `jdeinterop.ini` before deploying to production. See: Understanding jdeinterop.ini File.

JDE Atina Generate Configuration Files Tool - Using the Java CLI Application

It will require openjdk 8 installed.

Download the latest version (replace `1.0.0` with the latest one):

```
curl https://jfrog.atina-connection.com/artifactory/libs-release-local/com/atina/jd-
create-ini-files/1.0.0/jd-create-ini-files-1.0.0.jar \
--output jd-create-ini-files-1.0.0.jar
```

To check for the latest version, visit: jd-create-ini-files repository

Run the tool:

```
java -jar jd-create-ini-files-1.0.0.jar [OPTIONS]

OPTIONS:
    Options category 'startup':
      --debug [-d] (a string; default: "N")
        Debug Option
      --environment [-e] (a string; default: "")
        JDE Environment
      --password [-p] (a string; default: "")
        JDE Password for Enterprise Server Manager
      --server [-s] (a string; default: "")
        JDE URL of Server Manager
      --user [-u] (a string; default: "")
        JDE User for Enterprise Server Manager
```

Example

```
java -jar jd-create-ini-files-1.0.0.jar \
    -u jde_admin \
    -p XXXXXXX \
    -s http://server-manager:8999/manage \
    -e JDV920
```

JD Atina Generate Configuration Files Tool - Using the Docker Image

You can also run the same tool using Docker. This avoids needing Java locally.

```
docker run --rm -v /tmp/build_jde_libs:/tmp/build_jde_libs/ \
  -i --name jd-create-ini-files 92455890/jd-create-ini-files:1.0.0 \
  [user ex. jde_admin] \
  [password ex.secret123] \
  [environment ex JDV920] \
  [server ex. http://server:8999/manage]
```

Reviewing Output

```
Folder : /tmp/build_jde_libs/JPS920 has been created

Authenticating : http://server-manager:8999/manage
         Cookie:
044GXl43PnHkTe1L3AMc/siNkFOIoll+S4ngsdGnNkS7Qg=MDA5MDE1MDE1amRlX2FkbWluMTM0LjIwOS4yMTE
uMjQ4MTM0LjIwOS4yMTEuMjQ4MTYzNzYwNzAxMTIzOQ==
Getting Server Groups : http://server-manager:8999/manage

Select HTML Instance for environment JPS920:
0 - alpha_db
1 - alpha_dep
2 - alpha_dv920
3 - alpha_ent
4 - html_ps920
Q - Quit
4
Option Selected: 4

JDE Instance selected: html_ps920

Getting Instance Values: http://server-manager:8999/manage for Instance Name
'html_ps920'
 Processing File: jdbj.ini
 Processing File: jdeinterop.ini
 Processing File: jdelog.properties
 Processing File: settings.xml

------------------------------------------------------------------------
GENERATION SUCESSS
```

```
------------------------------------------------------------------------
 File: \tmp\build_jde_libs\JPS920\jdbj.ini generated
 File: \tmp\build_jde_libs\JPS920\jdeinterop.ini generated
 File: \tmp\build_jde_libs\JPS920\jdelog.properties generated
 File: \tmp\build_jde_libs\settings.xml generated
------------------------------------------------------------------------
```

The tool will generate:

```
build_jde_libs
       ├── settings.xml
       ├── JPS920
       ├──────────── ├── jdbj.ini
       ├──────────── ├── jdeinterop.ini
       └──────────── └── jdelog.properties
```

Adding manually "tnsnames.ora" for Oracle RDBMS based installations only.

JDE Atina Generate Jars Files Tool

This tool packages the JD Edwards libraries into deployable JAR files that are required by the Atina JDE Microservice. It simplifies dependency management and ensures that all necessary classes are available at runtime.

The tool will generate the following artifacts:

- `jde-lib-wrapped-1.0.0.jar`: Contains core JDE interoperability libraries.
- `StdWebService-1.0.0.jar`: Contains wrappers for standard JD Edwards web services.

You can use the tool either as a Java CLI or via Docker.

NOTE	Before running the tool, ensure you've copied the required files from your JD Edwards Deployment Server into the appropriate folder structure.

Required Folder Structure

Create the following directory layout and populate it with the required files:

```
/tmp/build_jde_libs/
 ├──── JDBC_Vendor_Drivers/         # From: //Deployment Server/E920/MISC/
 └──── system/
     ├──── Classes/                 # From: //Deployment Server/E920/system/Classes
     ├──── JAS/                     # From: //Deployment
Server/E920/system/JAS/webclient.ear/webclient.war/WEB-INF/lib/*
     └──── WS/                      # From: //Deployment
Server/E920/DV920/java/sbfjars
```

Define local Repository (localRepo option)

This option is required to run this tool using JAVA apps locally.

Running this command you can get where the curret local repository is defined:

```
mvn help:evaluate -Dexpression=settings.localRepository
```

In this output we see /root/.m2/repository

```
[INFO]
[INFO] ------------------< org.apache.maven:standalone-pom >-------------------
[INFO] Building Maven Stub Project (No POM) 1
[INFO] --------------------------------[ pom ]---------------------------------
[INFO]
[INFO] --- maven-help-plugin:3.2.0:evaluate (default-cli) @ standalone-pom ---
[INFO] No artifact parameter specified, using 'org.apache.maven:standalone-pom:pom:1'
as project.
[INFO]
/root/.m2/repository  <==  ****** THIS IS THE VALUE REQUIRED *******
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  3.301 s
[INFO] Finished at: 2021-11-22T20:31:42Z
[INFO] ------------------------------------------------------------------------
```

JDE Atina Generate Jars Files Tool - Using the Java CLI

It will require openjdk 8 installed.

Download the latest version:

```
curl https://jfrog.atina-connection.com/artifactory/libs-release-local/com/atina/jd-
create-jar-files/1.0.0/jd-create-jar-files-1.0.0-jar-with-dependencies.jar \
--output jd-create-jar-files-1.0.0-jar-with-dependencies.jar
```

Run the tool:

```
Usage: java -jar jd-create-jar-files-1.0.0-jar-with-dependencies.jar OPTIONS

Options category 'startup':
  --accion [-a] (a string; default: "3")
    Accion: 1: Generate JDE Bundle 2: Build WS 3: Both
  --clean [-c] (a string; default: "Y")
    clean
  --jdbcDriver [-j] (a string; default: '/tmp/build_jde_libs/JDBC_Vendor_Drivers')
```

```
    Enter JDBC Driver Folder
  --jdeInstallPath [-i] (a string; default: "/tmp/build_jde_libs/")
    Enter JDE Path installed
  --localRepo [-r] (a string; default: "")
    Enter Maven Local Repo
  --settings [-s] (a string; default: "/tmp/build_jde_libs/settings.xml")
    settings.xml to use Ex. /apache-maven-3.8.1/conf/settings.xml
  --version [-o] (a string; default: "1.0.0")
    Enter Version
```

Example:

```
java -jar jd-create-jar-files-1.0.0-jar-with-dependencies.jar -r /root/.m2/repository
```

JDE Atina Generate Jars Files Tool - Using Docker instead of Java Application

Run the following docker command:

```
docker run --rm -v /tmp/build_jde_libs:/tmp/build_jde_libs/ \
    -i --name jd-create-jar-files 92455890/jd-create-jar-files:1.0.0
```

Once completed, the following files will be created:

```
-------------------------------------------------------------------------
GENERATION SUCESSS
-------------------------------------------------------------------------
JDE Library bundle has been copied to: /tmp/build_jde_libs/jde-lib-wrapped-1.0.0.jar
JDE WS has been copied to: /tmp/build_jde_libs/StdWebService-1.0.0.jar
```

```
build_jde_libs
      ├── jde-lib-wrapped-1.0.0.jar
      └── StdWebService-1.0.0.jar
```

(Optional) Deploy Artifacts to a Local Maven Repository

Atina JDE Microservice can be configured to load its dependencies from a Maven-compatible repository. This optional step allows you to publish the generated JARs (`jde-lib-wrapped` and `StdWebService`) to a local or remote Maven repository such as JFrog Artifactory or Nexus.

This is useful if: - You want to automate dependency resolution during build or deployment - Your deployment environment does not allow copying files manually into the container

Step 1: Deploy `jde-lib-wrapped`

```
mvn deploy:deploy-file \
    -DgroupId=com.jdedwards \
    -DartifactId=jde-lib-wrapped \
    -Dversion=1.0.0 \
    -DrepositoryId=repo-central \
    -Dpackaging=jar \
    -Dfile=\tmp\build_jde_libs\jde-lib-wrapped-1.0.0.jar \
    -Durl=http://localrepo:8081/artifactory/libs-release
```

Step 2: Deploy `StdWebService`

```
mvn deploy:deploy-file \
    -DgroupId=com.jdedwards \
    -DartifactId=StdWebService \
    -Dversion=1.0.0 \
    -DrepositoryId=repo-central \
    -Dpackaging=jar \
    -Dfile=\tmp\build_jde_libs\StdWebService-1.0.0.jar \
    -Durl=http://localrepo:8081/artifactory/libs-release
```

NOTE Replace `repositoryId` and `url` with the values defined in your Maven `settings.xml`.

JDE Atina microservices - Installation

This section explains how to deploy the Atina JDE Microservice using Docker and Docker Compose. You'll configure environment variables, initialize the container, and run a basic test to validate connectivity with JD Edwards.

Step 1: Create the service-constants.properties File

In this step, you will create a configuration file that defines certain runtime parameters for BSFN services.

Using the terminal, create the following file:

```
mkdir -p /tmp/build_jde_libs
nano /tmp/build_jde_libs/service-constants.properties
```

Copy and paste the following content into the file:

```
J0100002_MAX_ROWS=10
J0000110_MAX_ROWS=10
```

```
J0100004_MAX_ROWS=10
J0400002_MAX_ROWS=10
J4200010_SOE_MBF_VERSION=
J4200010_BYPASS_WARNINGS=2
J4200010_PREFIX_5=ATINA
J4200010_PREFIX_2=ATINA
J4200050_MAX_ROWS=10
J0100022_MAX_ROWS=10
J0100007_MAX_ROWS=10
J4300010_PO_MBF_VERSION=ZJDE0001
J4300010_BYPASS_BSFN_WARNINGS=1
J4300010_PREFIX_1=
```

NOTE

Press `Ctrl + O`*, then* `Enter` *to save. Press* `Ctrl + X` *to exit.*

You may use any text editor you prefer, such as `vim`, `gedit`, or `nano`.

Step 3: Download the JDEAtinaOverrideServicesConstants Library

The system requires a supporting library to override default JD Edwards service properties with the values defined in your `service-constants.properties` file. This enables more flexible and file-driven configuration without modifying JDE directly.

Download the JAR file using the following command:

```
curl -L https://jfrog.atina-connection.com/artifactory/libs-release-
local/com/atina/JDEAtinaOverrideServicesConstants/1.0.0/JDEAtinaOverrideServicesConsta
nts-1.0.0.jar \
--output /tmp/build_jde_libs/JDEAtinaOverrideServicesConstants-1.0.0.jar
```

NOTE

Ensure this JAR file is located in the `/tmp/build_jde_libs/` folder along with the `service-constants.properties` file.

The system will automatically detect the presence of this library and use it to load web service configuration from the local file instead of querying JDE's internal property store.

Step 4: Download Docker Compose Files

Download the Docker Compose package:

```
curl https://jfrog.atina-connection.com/artifactory/libs-release/com/atina/jd-docker-
files/1.0.0/jd-docker-files-1.0.0.zip \
    --output jd-docker-files-1.0.0.zip
```

Unzip it:

```
7z e jd-docker-files-1.0.0.zip
```

Step 5: Configure Environment Variables

Edit the `.env` file and set the values according to your infrastructure:

Under **DNS_SERVERS** section complete these values:

```
#
# ====================================
# ETC/HOST
# ====================================
#
JDE_MICROSERVER_ENTERPRISE_SERVER_NAME=JDE-ENT
JDE_MICROSERVER_ENTERPRISE_SERVER_IP=1.1.1.1
JDE_MICROSERVER_ENTERPRISE_DB_NAME=JDE-DATABASE
JDE_MICROSERVER_ENTERPRISE_DB_IP=2.2.2.2
```

Step 6: Initialize the Container

Create the container without starting it:

```
docker-compose -f docker-compose-dist.yml up --no-start
```

Step 7: Copy Required Files into the Container

Copy the configuration and JAR files that were previously generated:

```
docker cp /tmp/build_jde_libs/service-constants.properties jd-atina-
microserver:/var/jdeatinaserver
docker cp /tmp/build_jde_libs/JPS920 jd-atina-microserver:/tmp/jde/config
docker cp /tmp/build_jde_libs/JDV920 jd-atina-microserver:/tmp/jde/config
docker cp JDEAtinaOverrideServicesConstants-1.0.0.jar jd-atina-microserver:/tmp/jde
docker cp /tmp/build_jde_libs/jde-lib-wrapped-1.0.0.jar jd-atina-microserver:/tmp/jde
docker cp /tmp/build_jde_libs/StdWebService-1.0.0.jar jd-atina-microserver:/tmp/jde
```

Step 8: Start the Container

```
docker-compose -f docker-compose-dist.yml start
```

Check the microservice startup log:

```
docker exec -it jd-atina-microserver cat /tmp/start.log
```

```
-SERVICE-----------------------------------------
   Name:  172.28.0.2
   Port:  8077
-REPOSITORY--------------------------------------
   Customer:  http:157.245.236.175:8081/artifactory/libs-release
   Atina:     http:157.245.236.175:8081/artifactory/libs-release
-LIBRARIES---------------------------------------
   StdWebService Version  1.0.0
   jde-lib-wrapped Version 1.0.0
   JDEAtinaServer Version 1.0.0
-LICENSE-----------------------------------------
   Code:  demo
-MICROSERVER-------------------------------------
   JDE_MICROSERVER_TOKEN_EXPIRATION:  3000000
   JDE_MICROSERVER_ENTERPRISE_SERVER_NAME:  JDE-ENT
   JDE_MICROSERVER_ENTERPRISE_SERVER_IP:  522.21.33.261
   JDE_MICROSERVER_ENTERPRISE_DB_NAME:  JDE-DATABASE
   JDE_MICROSERVER_ENTERPRISE_DB_IP:  625.22.339.57
   JDE_MICROSERVER_MOCKING:  0
-------------------------------------------------
ADDITIONAL SCRIPT:
  127.0.0.1       localhost
  172.28.0.2      bcd05e8e5bd0
-------------------------------------------------
 Check log cat /tmp/jde/JDEConnectorServerLog/jd_atina_server_2021-11-18.0.log
```

At the end of the logs, you will the log that you need to check. Run the following docker command:

```
docker exec -it jd-atina-microserver cat
/tmp/jde/JDEConnectorServerLog/jde_atina_server_2021-11-18.0.log
```

```
....
19:27:06.713 [main] INFO  c.acqua.jde.jdeconnectorserver.JDEConnectorServer -
Iniciando JDE Service Impl...
19:27:06.891 [main] INFO  c.a.jde.jdeconnectorserver.server.JDERestServer -
*-----------------------------------*
19:27:06.892 [main] INFO  c.a.jde.jdeconnectorserver.server.JDERestServer - *
Starting JDE Microservice 1.0.0   *
19:27:07.061 [main] INFO  c.a.jde.jdeconnectorserver.server.JDERestServer - *   JDE
Microservice started!         *
19:27:07.064 [main] INFO  c.a.jde.jdeconnectorserver.server.JDERestServer -
*-----------------------------------*
```

or if you prefer you can run the following command:

```
docker logs jd-atina-microserver
```

Validating the Microservice Deployment

Once the microservice is deployed and running, you can use the **JD Atina Check Microservice Tool** to validate that it can connect to JD Edwards and invoke a basic web service.

This tool is a standalone Java application that performs the following test:

- Logs in to JD Edwards using provided credentials
- Invokes the standard Address Book Web Service (`JP010000`)
- Logs out and displays the result

Step 1: Download the Check Tool

```
curl https://jfrog.atina-connection.com/artifactory/libs-release-local/com/atina/jd-
check-microservice/1.0.0/jd-check-microservice-1.0.0.jar \
    --jd-check-microservice-1.0.0.jar
```

Step 2: Run the Test

```
Usage: java -jar jd-check-microservice OPTIONS

   Options category 'startup':

  --mode [-m] (a string; default: "TestLoggindAndGetAddressBookWS")
    Modes
  --user [-u] (a string; default: "")
    JDE User
  --password [-w] (a string; default: "")
    JDE Password
  --environment [-e] (a string; default: "")
    JDE Environment
  --role [-r] (a string; default: "")
    JDE Role
  --serverName [-s] (a string; default: "")JD Micreserver Name or IP
  --serverPort [-p] (a string; default: "")
    JD Micreserver Port
  --addressbookno [-a] (a string; default: "")
    Address Book No
```

Usage Exampes

```
java -jar jd-check-microservice-1.0.0.jar -u JDE -w XXXXXX -e JDV920 -r *ALL -s
localhost -p 8077 -m TestLoggindAndGetAddressBookWS -a 28
```

Step 3: Expected Output

If everything is correctly configured, you will see output like the following:

```
Checking Microservice...
user=JDE, environment=JDV920, role=*ALL, serverName=192.168.99.100, serverPort=8077,
mode=TestLoggindAndGetAddressBookWS, addressbookno=1, sessionId=, transactionId=
Transaction ID: 20211122124518
User User: JDE in environment JDV920 with *ALL connected with Session ID [-1636010069]
WS JP010000.AddressBookManager.getAddressBook has been called correctly
Logout [0]
--------------------------------------------------------------------------
CHECK SUCESSS
--------------------------------------------------------------------------
User User: JDE in environment JDV920 with *ALL connected with Session ID -1636010069
Address Book Name: Financial/Distribution Company --
User User: JDE in environment JDV920 with *ALL disconnected. Current session ID 0
--------------------------------------------------------------------------
```